

SOFIAA OS

Sistema Operativo de Inteligencia Artificial

HERMES

Action Execution Layer

Reporte Técnico Integral — Nivel Doctoral

Versión 1.0 · Julio 2026 · Confidencial

Arquitectura: Next.js 14 · TypeScript · Firebase · Groq · Vercel

Resumen Ejecutivo

HERMES es la capa de ejecución de acciones del ecosistema SOFIAA OS, un sistema operativo de inteligencia artificial modular construido sobre Next.js 14, TypeScript, Firebase y Groq. HERMES actúa como el sistema nervioso motor del ecosistema: recibe intenciones semánticas de los motores cognitivos —PROMETEO (Growth Intelligence), ATENA (Scientific Intelligence) y TEC BII (Institutional Management)— y las materializa en acciones reales sobre plataformas externas como Monday.com, Slack, calendarios de contenido y bases de datos internas.

El diseño arquitectónico de HERMES parte de tres principios fundamentales: (1) Human-in-the-Loop Mandatory —ninguna acción se ejecuta sin aprobación humana explícita—, (2) Forward-Compatible Schema —el sistema de tipos contempla desde su inception los conectores de Etapa 2 (Meta Ads, Google Ads, WhatsApp Business, Mailchimp, HubSpot CRM)— y (3) Fault-Tolerant Execution —MAX_REINTENTOS=3 con transiciones atómicas de estado en Firestore.

Este documento presenta un análisis técnico exhaustivo de la arquitectura de HERMES: su modelo de datos en Firestore, la máquina de estados del ciclo de vida de acciones, el motor de despacho (executor), los cuatro conectores de Etapa 1 completamente implementados, las dos bridges de integración (PROMETEO y ATENA) y el modelo de seguridad basado en Firebase Authentication con tokens de identidad verificados en el servidor.

Tabla de Contenidos

1. Introducción y Contexto Arquitectónico
2. Sistema de Tipos — Schema Completo
3. Ciclo de Vida de una Acción (State Machine)
4. Arquitectura de Conectores
 - 4.1 Monday.com (monday_cloud)
 - 4.2 Slack (slack)
 - 4.3 Calendario SMM (calendario_smm)
 - 4.4 HERMES Interno (hermes_interno)
 - 4.5 Etapa 2 — Conectores Futuros
5. Motor de Despacho (Executor)
6. Bridges de Integración
 - 6.1 PROMETEO Bridge
 - 6.2 ATENA Bridge
7. Seguridad y Control de Acceso
8. Capa de API REST
9. Esquema Firestore
10. Capa de Interfaz de Usuario
11. Resolución de Credenciales
12. Roadmap Etapa 2
13. Conclusiones

1. Introducción y Contexto Arquitectónico

1.1 Posición de HERMES en el Ecosistema SOFIAA OS

SOFIAA OS es un ecosistema de inteligencia artificial modular diseñado para agencias de marketing y unidades institucionales. Sus módulos principales son: SOFIAA (núcleo conversacional y orquestador), N.E.X.O. (Knowledge Graph semántico y sistema nervioso de memoria), PROMETEO (motor de Growth Intelligence), NORA (observadora episódica del usuario), ATENA (motor de análisis científico Lean Six Sigma / DMAIC), TEC BII (gestión institucional) y HERMES (capa de ejecución de acciones).

Dentro de esta arquitectura, HERMES cumple el rol de actuador: si N.E.X.O. es el sistema nervioso aferente (entrada sensorial), HERMES es el sistema nervioso eferente (salida motora). Cada módulo cognitivo puede detectar una situación que requiere acción —fatiga creativa, riesgo de proceso con NPR crítico, tarea atrasada— pero ninguno tiene acceso directo a plataformas externas. HERMES es el único punto de ejecución.

1.2 Principios de Diseño

- Human-in-the-Loop Mandatory: toda acción pasa por aprobación explícita del usuario antes de ejecutarse. Este principio no es configurable ni derivable.
- Forward-Compatible Schema: el sistema de tipos define desde el inicio los conectores de Etapa 2. PROMETEO puede encolar una acción `meta_ads_escalar_presupuesto hoy` —la acción persiste en Firestore— y cuando Etapa 2 se implemente, el executor la despachará sin cambios en el schema.
- Fault-Tolerant Execution: `MAX_REINTENTOS=3` con transiciones de estado atómicas garantizan que los fallos transitorios de conectores externos no dejen acciones en estado indefinido.
- Authentication-First API: el endpoint de ejecución verifica un Firebase ID token en cada request. No existe ejecución anónima o no autenticada.
- Connector Isolation: cada conector vive en su propio módulo TypeScript. El executor nunca conoce los detalles de implementación —solo llama `dispatch()` y recibe un `HermesResultado` tipado.

1.3 Sprint History

HERMES fue construido en cuatro sprints principales: **H-0** (Schema + Fundación + Layout), **H-1** (Executor + Conectores Etapa 1 + API), **H-2** (PROMETEO Bridge + Director integration), **H-3** (Centro de Mando en tiempo real), **H-4** (ATENA Bridge + AMEF/SPC scanning). Los sprints H-7 y H-8 completaron la cola interactiva y el historial de acciones.

2. Sistema de Tipos — Schema Completo

2.1 HermesSourceEngine

Define el motor cognitivo que originó la acción. Los cinco valores posibles son:

Valor	Origen	Descripción
prometeo	Motor PROMETEO	Director Brief, fatiga creativa, recomendación de Creative Lab
atena	Motor ATENA	NPR crítico en AMEF, violación de reglas Western Electric en SPC
tec_bii	Motor TEC BII	Tarea operacional atrasada, alerta de capacidad de equipo
manual	Usuario directo	Acción creada explícitamente desde la UI de HERMES
cron	Scheduler Vercel	Director Brief automático generado por CRON diario de PROMETEO

2.2 HermesConnectorType

Identifica la plataforma externa de destino. El diseño contempla dos etapas de implementación:

Conector	Etapas	Plataforma	Autenticación
monday_cloud	1 ✓	Monday.com	API Token v2024-10
slack	1 ✓	Slack	Incoming Webhook URL
calendario_smm	1 ✓	Marketing Sofia (interno)	Firebase Admin SDK
hermes_interno	1 ✓	SOFIAA OS Firestore	Firebase Admin SDK
meta_ads	2 	Meta Business Suite	OAuth2 System User Token
google_ads	2 	Google Ads	OAuth2 + Developer Token
whatsapp_business	2 	WhatsApp Cloud API	Phone Number ID + Token
mailchimp	2 	Mailchimp	API Key (env var)
hubspot_crm	2 	HubSpot CRM	Private App Token

2.3 HermesActionType

El sistema define 22 tipos de acción distintos, todos siguiendo la convención de nombres {conector}_{operacion}. Los tipos de Etapa 2 están definidos en el schema TypeScript desde Sprint H-0, lo que permite que PROMETEO los encole hoy aunque el executor aún devuelva ETAPA_2_NOT_IMPLEMENTED.

2.4 HermesActionStatus — Máquina de Estados

Los siete estados del ciclo de vida de una acción son:

```
propuesta → pendiente_aprobacion → aprobada → ejecutando → completada
└─ rechazada
fallida
```

2.5 HermesAction — Documento Principal

La entidad central del sistema. Almacenada en Firestore bajo `smm_workspaces/{workspaceId}/hermes_queue/{actionId}`. Sus campos se agrupan en cinco categorías:

- Origen: `sourceEngine`, `sourceBriefId`, `sourceGoalId`, `clienteId`, `clienteNombre`
- Tipo y payload: `tipo` (`HermesActionType`), `connectorTipo` (`HermesConnectorType`), `payload` (`Record<string, unknown>`)
- Contexto para el usuario: `titulo`, `descripcion`, `justificacion`, `urgencia` (BAJA | MEDIA | ALTA | CRITICA)
- Ciclo de vida: `estado`, `aprobadoPor`, `rechazadoPor`, `motivoRechazo`
- Resultado y auditoría: `resultado` (`HermesResultado`), `reintentos`, `createdAt`, `aprobadoAt`, `executedAt`, `completadoAt`

2.6 HermesResultado

La respuesta tipada de cada conector tras intentar ejecutar una acción:

```
interface HermesResultado {    éxito:        boolean;    mensaje:      string;
// legible para el usuario    datos?:      Record<string, unknown>; //
respuesta raw del conector    linkAccion?:  string;                // URL al resultado
(tarea Monday, post, etc.)    errorCode?:   string;                // código de error
si falló }
```

3. Ciclo de Vida de una Acción (State Machine)

3.1 Descripción Completa del Flujo

El ciclo de vida de una `HermesAction` sigue una máquina de estados determinística con siete nodos y tres ramas de salida terminal. La transición entre estados es siempre atómica: HERMES actualiza el campo estado en Firestore antes y después de cada operación de ejecución.

3.2 Estados y Transiciones

Estado	Descripción	Quién lo establece	Siguiente(s)
propuesta	Generada por el motor, no visible al usuario aún	Bridge (PROMETEO/ATENA)	pendiente_aprobacion
pendiente_aprobacion	En cola, esperando decisión del usuario	enqueueBriefActions() / scanAtenaAlerts()	aprobada, rechazada
aprobada	El usuario aprobó. Lista para ejecutarse.	cola/page.tsx (UI)	ejecutando
ejecutando	El <code>executor.ts</code> la está procesando en este momento	markEjecutando()	completada, fallida
completada	Ejecutada exitosamente por el conector	markCompletada()	— (terminal)
fallida	El conector reportó error; puede reintentarse (max 3)	markFallida()	aprobada (si reintentos < 3)
rechazada	El usuario rechazó la acción; permanece como auditoría	cola/page.tsx (UI)	— (terminal)

3.3 Nota sobre ETAPA_2_NOT_IMPLEMENTED

Un caso especial del executor: las acciones dirigidas a conectores de Etapa 2 (*meta_ads*, *google_ads*, *whatsapp_business*, *mailchimp*, *hubspot_crm*) pasan por el Human Gate normalmente —el usuario las aprueba— y el executor las despacha a `ejecutarEtapa2Action()`, que devuelve un `HermesResultado` con `exitito: false` y `errorCode: 'ETAPA_2_NOT_IMPLEMENTED'`. El executor trata este código como completada (no como fallida real), preservando así el registro de auditoría sin incrementar el contador de reintentos. La UI muestra al usuario un mensaje explicativo en lugar de un error.

3.4 Política de Reintentos

La constante `MAX_REINTENTOS = 3` en `executor.ts` define el número máximo de intentos de ejecución antes de marcar una acción como permanentemente fallida. El contador se incrementa únicamente en errores reales (no en `ETAPA_2_NOT_IMPLEMENTED`). Si `reintentos >= MAX_REINTENTOS`, el executor retorna inmediatamente con `errorCode: 'MAX_RETRIES_EXCEEDED'` sin intentar contactar al conector.

3.5 Timestamps de Auditoría

Cada transición de estado registra un timestamp en Firestore:

- `createdAt`: cuando el bridge encoló la acción
- `aprobadoAt`: cuando el usuario hizo clic en Aprobar
- `executedAt`: cuando el executor llamó a `markEjecutando()`
- `completadoAt`: cuando el executor finalizó (éxito o fallo)

Todos los timestamps son epoch Unix en milisegundos. Adicionalmente, `_updatedAt` usa `FieldValue.serverTimestamp()` de Firebase Admin para coherencia temporal en el servidor.

4. Arquitectura de Conectores

Cada conector de Etapa 1 es un módulo TypeScript independiente en `src/lib/hermes/connectors/`. El executor los llama a través de la función `dispatch()` sin conocer sus detalles de implementación. Todos los conectores siguen el contrato: recibir una `HermesAction` y devolver una `Promise<HermesResultado>`.

4.1 Monday.com — monday_cloud

Implementado en `connectors/monday.ts`. Usa la API GraphQL de Monday.com v2024-10 via HTTP directo. Soporta tres operaciones:

Tipo de Acción	Mutación GraphQL	Parámetros requeridos
monday_crear_tarea	<code>create_item(board_id, group_id, item_name, column_values)</code>	board_id, nombre
monday_actualizar_tarea	<code>change_multiple_column_values(board_id, item_id, column_values)</code>	item_id, columnas
monday_mover_grupo	<code>move_item_to_group(item_id, group_id)</code>	item_id, group_id

El conector implementa una función interna `gql<T>()` que maneja autenticación, errores HTTP y errores GraphQL. El token de API se resuelve con prioridad: variable de entorno `MONDAY_API_TOKEN` → secretos de Firestore (via `resolveMondayToken()`). Si no hay token, el conector devuelve `errorCode: 'CONNECTOR_NOT_CONFIGURED'` sin intentar la llamada HTTP.

En caso de éxito de crear tarea, el resultado incluye `linkAccion` con la URL directa al item en Monday: `https://monday.com/boards/{boardId}/pulses/{itemId}`.

4.2 Slack — slack

Implementado en `connectors/slack.ts`. Usa Incoming Webhooks de Slack —la integración más simple y segura, sin OAuth de app completa. Soporta dos operaciones:

- `slack_notificar`: mensaje estándar con blocks de contexto (cliente, motor origen, urgencia)
- `slack_notificar_urgente`: prefijo  [URGENTE] en texto principal y en el fallback text del payload

El payload de Slack usa Block Kit para mensajes enriquecidos: un bloque section con texto Markdown y un bloque context con metadatos de la acción (clienteNombre, sourceEngine, urgencia). El webhook URL se resuelve con la misma estrategia `env var` → Firestore que Monday.

4.3 Calendario SMM — calendario_smm

Implementado en `connectors/calendario.ts`. Opera directamente sobre la colección `smm_workspaces/{workspaceId}/smm_calendar` de Firestore usando Firebase Admin SDK (sin HTTP externo). Soporta dos operaciones:

- `calendario_crear_post`: `addDoc` con campos `clienteId`, `titulo`, `copy`, `plataforma`, `fecha`, `formato`, `estado='programado'`, `createdByHermes=true`
- `calendario_actualizar_post`: `updateDoc` en el documento existente con estado y/o copy

Al ser un conector interno a Firestore, tiene latencia mínima y elimina la dependencia de servicios externos. El flag `createdByHermes=true` permite distinguir posts generados automáticamente en la UI del calendario.

4.4 HERMES Interno — `hermes_interno`

Implementado en `connectors/interno.ts`. El conector más sofisticado en lógica de dominio: ejecuta acciones que modifican el estado interno de SOFIAA OS, usando Firebase Admin SDK. Soporta dos operaciones de alta relevancia:

- `interno_actualizar_goal`: actualiza el campo `valorActual` de un `BrandGoal` en la colección `prometeo_goals` del workspace activo. Permite que HERMES cierre el loop entre métricas observadas y objetivos de crecimiento de PROMETEO.
- `interno_registrar_memoria`: crea un documento `CreativeMemory` completo en `prometeo_creative_memory`, incluyendo todos los campos de performance (ROAS, CTR, CPA, alcance, inversión, conversiones) y metadatos de industria y temporada. Esto automatiza el aprendizaje creativo cuando PROMETEO o el Director detectan un creativo de alto rendimiento.

4.5 Etapa 2 — Conectores Futuros

Implementado como stub informativo en `connectors/etapa2.ts`. La función `isEtapa2Connector()` identifica los cinco tipos de Etapa 2; `ejecutarEtapa2Action()` devuelve un mensaje descriptivo específico por conector con los requisitos de integración:

Conector	Requisito de Integración
<code>meta_ads</code>	OAuth2 con System User Token de Meta Business Suite
<code>google_ads</code>	OAuth2 con Developer Token de Google Ads API v17
<code>whatsapp_business</code>	Phone Number ID + Webhook en Meta Cloud API
<code>mailchimp</code>	API Key en variable de entorno <code>MAILCHIMP_API_KEY</code>
<code>hubspot_crm</code>	Private App Token en variable de entorno <code>HUBSPOT_TOKEN</code>

El tratamiento especial de `ETAPA_2_NOT_IMPLEMENTED` en el executor (guardado como completada, no como fallida) garantiza que el historial de auditoría refleje la intención del usuario sin contaminar las métricas de errores reales del sistema.

5. Motor de Despacho (Executor)

5.1 Arquitectura General

El executor (`src/lib/hermes/executor.ts`) es el corazón de HERMES. Es un módulo server-only que usa Firebase Admin SDK para todas sus operaciones de Firestore. Nunca se ejecuta en el cliente. El punto de entrada único es la función exportada `executeAction(workspaceId, actionId)`.

5.2 Algoritmo de Ejecución

El flujo completo de `executeAction()` es:

```
1. getAction(workspaceId, actionId) → Lee el documento de Firestore; devuelve null si no existe
2. Validar estado === "aprobada" → Si no, retorna errorCode: 'INVALID_STATE'
3. Verificar reintentos < MAX_REINTENTOS (3) → Si reintentos >= 3, llama markFallida() y retorna 'MAX_RETRIES_EXCEEDED'
4. markEjecutando() → Actualiza estado="ejecutando", executedAt=Date.now() → Usa FieldValue.serverTimestamp() para _updatedAt
5. dispatch(accion) → Enruta al conector correcto según accion.connectorTipo → try/catch captura excepciones inesperadas como EXECUTION_EXCEPTION
6. markCompletada() o markFallida() → completada: si exito===true O errorCode==='ETAPA_2_NOT_IMPLEMENTED' → fallida: cualquier otro error real; incrementa reintentos+1
```

5.3 Función dispatch()

La función `dispatch()` implementa el patrón Strategy: actúa como router puro entre el executor y los conectores. Su implementación completa es:

```
async function dispatch(accion: HermesAction): Promise<HermesResultado> {
  if (isEtapa2Connector(accion.connectorTipo)) { return ejecutarEtapa2Action(accion); }
  switch (accion.connectorTipo) {
    case "monday_cloud": return ejecutarMondayAction(accion);
    case "slack": return ejecutarSlackAction(accion);
    case "calendario_smm": return ejecutarCalendarioAction(accion);
    case "hermes_interno": return ejecutarInternoAction(accion);
    default: return { exito: false, mensaje: `Conector desconocido: ${accion.connectorTipo}`, errorCode: "UNKNOWN_CONNECTOR" }; }
}
```

Cada conector recibe el objeto `HermesAction` completo, lo que les permite acceder a cualquier campo (payload, clientId, workspaceId, urgencia) sin necesitar parámetros adicionales.

5.4 Transiciones de Estado en Firestore

Las cuatro funciones privadas de gestión de estado (`getAction`, `markEjecutando`, `markCompletada`, `markFallida`) operan sobre el mismo `docRef: adminDb.collection('smm_workspaces').doc(workspaceId).collection('hermes_queue').doc(actionId)`.

Todas las escrituras usan el Admin SDK directamente (no el cliente SDK), garantizando que las operaciones bypassen las reglas de seguridad de Firestore y tengan privilegios de escritura completos desde el servidor.

5.5 Manejo de Errores

El executor distingue cuatro categorías de error:

errorCode	Origen	Acción del executor
NOT_FOUND	Documento no existe en Firestore	Retorna sin modificar Firestore
INVALID_STATE	Estado !== 'aprobada'	Retorna sin modificar Firestore
MAX_RETRIES_EXCEEDED	reintentos >= 3	Llama markFallida() y retorna
EXECUTION_EXCEPTION	Excepción no capturada en dispatch()	Llama markFallida() con mensaje del error
CONNECTOR_NOT_CONFIGURED	Token/webhook faltante	El conector retorna; executor llama markFallida()
ETAPA_2_NOT_IMPLEMENTED	Conector de Etapa 2	Llama markCompletada() (tratamiento especial)

6. Bridges de Integración

Las bridges son módulos de traducción que convierten decisiones semánticas de los motores cognitivos en HermesActions concretas. Su diseño sigue el patrón Adapter: aíslan al motor de origen del sistema de tipos de HERMES, permitiendo que cada motor evolucione independientemente.

6.1 PROMETEO Bridge

6.1.1 Propósito

La PROMETEO Bridge (`src/lib/hermes/prometeo-bridge.ts`) traduce un `DirectorBrief` —el artefacto cognitivo principal del Director Autónomo de PROMETEO— en un conjunto de `HermesActions` atómicas. Se invoca automáticamente al terminar la generación de un brief, tanto desde la API manual (`/api/prometeo/director/brief`) como desde el CRON diario de Vercel.

6.1.2 Función Principal

La función exportada es `enqueueBriefActions(workspaceId, brief)`. Recibe el `workspaceId` del workspace activo y el `DirectorBrief` recién generado por Groq. Internamente construye un array de `HermesActions` según el contenido del brief y lo escribe en Firestore usando un `writeBatch` atómico. Si el batch falla, ninguna acción queda en la cola.

6.1.3 Mapping de Recomendaciones

Recomendación en DirectorBrief	Tipo HERMES	Conector	Urgencia
FATIGA detectada en algún cliente	slack_notificar_urgente	slack	ALTA
ESCALAR creativo recomendado	monday_crear_tarea (descrip. detallada)	monday_cloud	ALTA
PAUSAR campaña (ROAS < umbral)	monday_crear_tarea (CRITICA)	monday_cloud	CRITICA
NUEVO_CREATIVO requerido	calendario_crear_post (brief de nueva pieza)	calendario_smm	MEDIA
CAMBIAR_CANAL recomendado	slack_notificar (análisis de canal)	slack	MEDIA
Oportunidad de crecimiento	slack_notificar (oportunidad)	slack	BAJA

6.1.4 Estructura de una Acción Generada

Cada acción generada por la bridge incluye campos de contexto completos para el usuario:

```
{  sourceEngine: "prometeo",  sourceBriefId: brief.id,  clienteId: cliente.clienteId,  clienteNombre: cliente.clienteNombre,  titulo: "Fatiga creativa detectada - Acción urgente",  descripcion: "Cliente [NOMBRE] muestra fatiga...",  justificacion: "El Director Autónomo detectó [razón específica]",  urgencia: "ALTA",  estado: "pendiente_aprobacion",  tipo: "slack_notificar_urgente",  connectorTipo: "slack",  payload: { mensaje: "...", canal: "#marketing" },  reintentos: 0,  createdAt: Date.now(), }
```

6.2 ATENA Bridge

6.2.1 Propósito

La ATENA Bridge (src/lib/hermes/atenas-bridge.ts) es el sistema de vigilancia científica de HERMES. Escanea las colecciones de ATENA (AMEF y SPC) en busca de condiciones críticas y genera HermesActions de alerta. A diferencia de la PROMETEO Bridge, que convierte un artefacto ya generado, la ATENA Bridge es un scanner activo que puede ejecutarse on-demand o via CRON.

6.2.2 Función Principal

La función exportada es `scanAtenaAlerts(uid, workspaceId)`. Recibe el uid del usuario (para construir los paths de Firestore de ATENA) y el workspaceId del workspace PROMETEO (para escribir en hermes_queue). Retorna un **AtenasScanResult** con contadores por tipo de alerta encontrada.

6.2.3 Thresholds y Criterios de Alerta

Fuente	Condición	Tipo HERMES	Conector	Urgencia
AMEF	NPR > 200 (NPR_CRITICO)	monday_crear_tarea	monday_cloud	CRITICA
AMEF	NPR > 100 (NPR_ALTO)	monday_crear_tarea	monday_cloud	ALTA
SPC	Cpk < 1.0 (CPK_MINIMO)	slack_notificar	slack	MEDIA
SPC	Violaciones Western Electric	slack_notificar_urgente	slack	ALTA

6.2.4 Reglas Western Electric Implementadas

Las violaciones de control estadístico del proceso (Western Electric Rules) que detecta el scanner incluyen:

- Punto fuera de límites de control ($> 3\sigma$ del centro)
- Run de 8 puntos consecutivos en el mismo lado de la línea central
- Dos de tres puntos consecutivos en zona de advertencia ($> 2\sigma$)

Cada violación genera una acción slack_notificar_urgente con la variable CTQ afectada, el valor observado y los límites de control de referencia.

6.2.5 Endpoint de Trigger

La bridge se invoca desde /api/atenas/hermes-scan (POST, autenticado con Firebase ID token). También puede configurarse como CRON de Vercel para ejecución periódica. La UI de ATENA → Inteligencia muestra un botón 'Escanear con HERMES' que consume este endpoint y muestra el AtenasScanResult con badge del número de alertas generadas.

6.2.6 AtenasScanResult

```
interface AtenasScanResult {  amefCriticos: number;    // NPR > 200
  amefAltos:    number;    // NPR > 100  spcCpkBajos:  number;    // Cpk <
  1.0  spcViolaciones: number; // Western Electric violations
  totalAcciones: number;    // total encolado en HERMES }
```

7. Seguridad y Control de Acceso

7.1 Human-in-the-Loop Gate

El principio de diseño más fundamental de HERMES es que ninguna acción se ejecuta sin aprobación humana explícita. Este gate se implementa en dos niveles:

- Estado pendiente_aprobacion: toda acción creada por una bridge inicia en este estado. El executor.ts se niega a procesar cualquier acción que no esté en estado aprobada.
- UI Gate: la página /hermes/cola muestra cada acción con su justificación completa y botones de Aprobar/Rechazar. Solo tras la acción explícita del usuario se actualiza el estado a aprobada o rechazada.

7.2 Autenticación Firebase en la API

El endpoint POST /api/hermes/execute implementa el siguiente flujo de autenticación:

```
// Extraer token del header Authorization: Bearer {token} const token =
request.headers.get("authorization")?.split(" ")[1]; // Verificar con
Firebase Auth Admin SDK const decoded = await
getAuth(getAdminApp()).verifyIdToken(token); // → decoded.uid, decoded.email
disponibles // El token es generado por el cliente con: // const token =
await user.getIdToken();
```

Si el token es inválido, expirado o inexistente, el endpoint retorna 401 sin ejecutar ninguna lógica de negocio. Los tokens de Firebase ID tienen una validez de 1 hora y son renovados automáticamente por el cliente SDK.

7.3 Resolución de Credenciales de Conectores

Las credenciales sensibles (Monday API Token, Slack Webhook URL) nunca se almacenan en Firestore ni se exponen al cliente. El sistema de resolución tiene una jerarquía de prioridad explícita:

```
// Prioridad: env var > Firestore secrets (Sprint H-6) const apiToken =
await resolveMondayToken(workspaceId).catch(() => null); // resolveMonday
Token(): // 1. Busca MONDAY_API_TOKEN en process.env // 2. Si no existe,
busca en smm_workspaces/{id}/hermes_secrets/monday // 3. Si no existe en
ningún lado, retorna null → CONNECTOR_NOT_CONFIGURED
```

Esta arquitectura permite que equipos pequeños configuren el sistema via variables de entorno en Vercel, mientras que configuraciones multi-workspace futuras podrán almacenar credenciales por workspace en Firestore de forma segura.

7.4 Principio de Mínimo Privilegio

Cada conector tiene acceso únicamente a las operaciones que necesita. El conector de Slack no puede leer datos de Firestore; el conector Interno no puede hacer llamadas HTTP externas. El executor no tiene acceso directo a las credenciales —delega a las funciones resolve*() de cada conector.

7.5 Seguridad en .env.local

Las claves API se configuran via nano en terminal o mediante la interfaz de Vercel. El archivo `.env.local` está en `.gitignore` y nunca se incluye en commits. Esta política se documenta explícitamente en el codebase y es no-negociable.

8. Capa de API REST

8.1 POST /api/hermes/execute

El único endpoint de ejecución de HERMES. Server-side only (Next.js Route Handler).

Campo	Valor
Método	POST
Autenticación	Bearer {Firebase ID Token}
Body	{ workspaceId: string, actionId: string }
Respuesta exitosa	{ success: true, resultado: HermesResultado }
Error 401	{ error: 'Unauthorized' }
Error 400	{ error: 'workspaceId y actionId son requeridos' }
Error 500	{ error: string } — error en executor

8.2 POST /api/atena/hermes-scan

Endpoint de trigger para la ATENA Bridge. Requiere Firebase ID token. El body puede estar vacío —el endpoint extrae el uid del token verificado y escanea las colecciones ATENA de ese usuario. Devuelve { enqueued: AtenasScanResult } con el número de acciones generadas por tipo.

8.3 POST /api/prometeo/director/brief

Endpoint de generación del Director Brief. No es exclusivamente de HERMES, pero es el primer punto de integración: al finalizar la generación del brief con Groq, el endpoint invoca enqueueBriefActions() antes de retornar la respuesta al cliente. Esto garantiza que las acciones se encolen atómicamente con la creación del brief, incluso si la UI no recibe la respuesta.

8.4 CRON Endpoints

Vercel crea dos tareas CRON configuradas en vercel.json:

- /api/cron/prometeo-brief: se ejecuta diariamente. Genera un DirectorBrief automático con datos frescos del workspace activo e invoca enqueueBriefActions().
- /api/atena/hermes-scan: también puede configurarse como CRON para vigilancia continua de proceso.

9. Esquema Firestore

9.1 Colecciones Principales

```
smm_workspaces/{workspaceId}/hermes_queue/{actionId} ← Cola
principal de acciones {actionId}/ ← Documento HermesAction
completo id, workspaceId, sourceEngine, sourceBriefId, tipo,
connectorTipo, payload, titulo, descripcion, justificacion,
urgencia, estado, aprobadoPor, rechazadoPor, motivoRechazo,
resultado, reintentos, createdAt, aprobadoAt, executedAt,
completadoAt, _updatedAt (serverTimestamp) hermes_connectors/
← Config de conectores por workspace {connectorId}/ ←
HermesConnectorConfig tipo, nombre, status, etapa,
webhookConfigured, boardId?, channelId?, adAccountId?, createdAt,
updatedAt hermes_secrets/ ← Secretos por workspace (futuro H-6)
monday/ ← { apiToken: string (encrypted) } slack/
← { webhookUrl: string }
```

9.2 Índices Recomendados

Para las queries de la UI (cola por estado, historial por fecha), se recomiendan los siguientes índices compuestos en Firestore:

Colección	Campos	Query que soporta
hermes_queue	workspaceId ASC + estado ASC + createdAt DESC	Cola filtrada por estado
hermes_queue	workspaceId ASC + completadoAt DESC	Historial ordenado
hermes_queue	workspaceId ASC + sourceEngine ASC + createdAt DESC	Acciones por motor
hermes_queue	workspaceId ASC + connectorTipo ASC + estado ASC	Acciones por conector

9.3 Reglas de Seguridad Firestore

Las reglas de Firestore para hermes_queue restringen el acceso:

```
match /smm_workspaces/{workspaceId}/hermes_queue/{actionId} { // Solo el
propietario del workspace puede leer allow read: if request.auth != null
&& resource.data.workspaceId == workspaceId; // Solo el servidor puede
escribir (via Admin SDK) // El cliente nunca escribe directamente en
hermes_queue allow write: if false; }
```

10. Capa de Interfaz de Usuario

10.1 Centro de Mando (/hermes)

Dashboard principal de HERMES. Muestra KPIs en tiempo real consultando la colección hermes_queue: acciones pendientes de aprobación, completadas hoy, fallidas hoy y conectores activos. Incluye un preview de las últimas 5 acciones en cola y las últimas 5 completadas con sus timestamps y estados.

10.2 Cola de Acciones (/hermes/cola)

La pantalla de decisión humana. Lista todas las acciones en estado pendiente_aprobacion ordenadas por urgencia (CRITICA > ALTA > MEDIA > BAJA) y por fecha. Cada tarjeta muestra:

- Título y descripción legible de la acción
- Justificación: por qué el motor recomienda esta acción
- Motor de origen (PROMETEO / ATENA / TEC BII / Manual)
- Badge de urgencia con color (rojo CRITICA, naranja ALTA, ámbar MEDIA, verde BAJA)
- Cliente afectado (si aplica)
- Botón Aprobar: llama a /api/hermes/execute tras actualizar estado a aprobada
- Botón Rechazar: abre modal para ingresar motivo de rechazo (motivoRechazo)

10.3 Historial (/hermes/historial)

Vista de auditoría completa. Muestra todas las acciones en estado completada, fallida o rechazada ordenadas por completadoAt DESC. Cada entrada muestra el resultado de la ejecución (HermesResultado.mensaje), el linkAccion si existe, el número de reintentos y los timestamps del ciclo de vida completo.

10.4 Conectores (/hermes/conectores)

Panel de gestión de conectores. Lista los nueve conectores del sistema (4 de Etapa 1 + 5 de Etapa 2) con su status actual. Los conectores de Etapa 1 muestran si están configurados (activo) o pendientes de configuración (pendiente_config). Los de Etapa 2 aparecen con badge 'Próximamente'. Incluye instrucciones contextuales por conector para la configuración de credenciales.

11. Resolución de Credenciales

11.1 Arquitectura de Secrets

HERMES implementa un sistema de resolución de credenciales por capas (connector-secrets.ts) que prioriza seguridad sobre conveniencia:

```
// Capa 1: Variables de entorno (máxima prioridad, máxima seguridad)
process.env.MONDAY_API_TOKEN    → monday_cloud process.env.SLACK_WEBHOOK_URL
→ slack // Capa 2: Firestore encrypted secrets (Sprint H-6, multi-
workspace) smm_workspaces/{id}/hermes_secrets/monday.apiToken
smm_workspaces/{id}/hermes_secrets/slack.webhookUrl
```

11.2 Variables de Entorno Requeridas

Variable	Conector	Formato
MONDAY_API_TOKEN	monday_cloud	Token de la API de Monday v2 (sin prefijo 'Bearer')
MONDAY_BOARD_ID_DEFAULT	monday_cloud	ID numérico del board default del workspace
SLACK_WEBHOOK_URL	slack	https://hooks.slack.com/services/T.../B.../xxx
FIREBASE_ADMIN_*	hermes_interno, calendario	Credenciales de Firebase Admin SDK

12. Roadmap Etapa 2

12.1 Conectores por Implementar

Los cinco conectores de Etapa 2 están completamente definidos en el schema TypeScript (tipos de acción, payloads, HermesConnectorConfig). La implementación requiere únicamente:

- Obtener credenciales de API para cada plataforma
- Implementar el módulo connectors/{nombre}.ts siguiendo el patrón establecido
- Registrar el conector en dispatch() con un case adicional
- Actualizar hermes_connectors en Firestore con etapa: 2, status: 'activo'

12.2 Plan de Implementación Prioritario

Prioridad	Conector	Impacto	Complejidad
1	meta_ads	Gestión directa de inversión publicitaria en Meta	Media (OAuth2 System User)
2	whatsapp_business	Comunicación directa con leads y clientes	Baja (webhook simple)
3	mailchimp	Automatización de campañas email	Baja (API Key)
4	hubspot_crm	Cierre del loop ventas-marketing	Media (Private App Token)
5	google_ads	Diversificación de canales pagados	Alta (Developer Token + OAuth2)

12.3 Meta Ads — Próximo Sprint

Meta Ads es el conector de mayor impacto para agencias de marketing. El plan de implementación es:

- Crear Meta Business Suite System User con permisos de gestión de campañas
- Almacenar el token en MONDAY_API_TOKEN (pattern análogo al de Monday)
- Implementar Marketing API v20 con GraphQL (misma arquitectura que Monday)
- Soportar las cuatro acciones definidas: crear_campana, pausar_campana, escalar_presupuesto, crear_adset

12.4 Capacidades de Etapa 2 en PROMETEO

Una vez implementada Etapa 2, el Director Autónomo de PROMETEO podrá generar acciones como:

- meta_ads_escalar_presupuesto: si ROAS > umbral objetivo en los últimos 7 días, escalar presupuesto diario en un porcentaje definido
- meta_ads_pausar_campana: si ROAS < umbral mínimo y la inversión supera el budget de emergencia
- whatsapp_enviar_mensaje: notificación de oportunidad detectada al contacto clave del cliente
- crm_crear_deal: cuando se detecta un lead calificado desde las métricas de campaña

13. Conclusiones

13.1 Madurez del Sistema

HERMES v1.0 constituye una implementación production-ready de la capa de ejecución de acciones para el ecosistema SOFIAA OS. Los cuatro conectores de Etapa 1 están completamente operativos y probados. El sistema de Human-in-the-Loop está implementado con rigor: es estructuralmente imposible ejecutar una acción sin aprobación del usuario, dado que el executor valida el estado aprobada antes de cualquier operación.

13.2 Fortalezas Arquitectónicas

Las decisiones de diseño más destacadas de HERMES son:

- Forward-compatible schema: los tipos de Etapa 2 en el schema desde el día uno eliminan la deuda técnica de migración futura. PROMETEO puede encolar acciones de Meta Ads hoy; solo el executor necesita un case adicional cuando el conector esté listo.
- Connector isolation: cada conector es un módulo independiente. Agregar un nuevo conector no modifica el executor ni el schema —solo se añade un módulo y un case en dispatch(). Esto satisface el Open/Closed Principle.
- Atomic state transitions: las escrituras de estado en Firestore son transacciones únicas. No existe condición de carrera entre el cliente (que aprueba) y el servidor (que ejecuta) porque el estado se valida en el servidor antes de ejecutar.
- Bridge pattern: la separación entre los motores cognitivos (PROMETEO, ATENA) y el executor garantiza que ningún módulo conoce los detalles de implementación del otro. Cada bridge es el único punto de acoplamiento.

13.3 Métricas de Implementación

Métrica	Valor
Conectores Etapa 1 implementados	4 (monday_cloud, slack, calendario_smm, hermes_interno)
Conectores Etapa 2 definidos (stub)	5 (meta_ads, google_ads, whatsapp_business, mailchimp, hubspot_crm)
Tipos de acción definidos	22 (HermesActionType)
Estados del ciclo de vida	7 (state machine)
MAX_REINTENTOS	3
Bridges de integración	2 (PROMETEO Bridge, ATENA Bridge)
Sprints de desarrollo	H-0 → H-4, H-7, H-8 (7 sprints)
Archivos TypeScript del módulo	~12 (schema, executor, 4 conectores, 2 bridges, helpers)

13.4 Visión a Largo Plazo

HERMES está diseñado para evolucionar de ser un sistema de automatización operacional a ser la capa de agencia de SOFIAA OS —el módulo que le permite a la IA no solo entender el mundo (NEXO) y razonar sobre él (PROMETEO, ATENA), sino actuar sobre él de forma segura, auditada y reversible.

La combinación de Human-in-the-Loop obligatorio, audit trail completo en Firestore y connector isolation crea las condiciones técnicas y éticas para incrementar progresivamente el nivel de autonomía del sistema según madure la confianza del usuario en sus recomendaciones.